

CSE 753: Advanced Reinforcement Learning

Contents

1	Policy Gradient Methods	2
---	-------------------------	---

1 Policy Gradient Methods

1.1 Generalized Policy Iteration

Generalized Policy Iteration — the core idea behind how an AI agent learns to make better decisions over time.

The Big Idea

The agent follows a simple two-step loop, repeated over and over:

1. **Collect experience** — Run the current policy (the agent’s current strategy, called π) out in the world and record what happened. These recorded sequences of actions and outcomes are called **trajectories** — basically a replay of what the agent did from start to finish.
2. **Improve the policy** — Look at all those trajectories. Which ones led to good outcomes? Which led to bad ones? Use that information to update the policy so it’s slightly smarter than before.

Then repeat. That’s it. The whole loop.

What “Initialize” Means

Before the loop starts, you need to give the agent some starting policy. You have three options:

- **Random** — just pick actions randomly at first (agent knows nothing)
- **Imitation learning** — copy how an expert does it
- **Heuristics** — use simple hand-coded rules as a starting point

How It Improves Over Time

The diagram on the right shows three snapshots — Iteration 1, 50, and 100. Think of each curved line as one trajectory (one attempt the agent made from start to finish).

- Red X = bad outcome (low reward)
- Yellow check = okay outcome
- Green check = good outcome

Stage	What’s happening
Early (Iter 1)	Most attempts fail. The agent is basically guessing. Lots of exploration.
Mid (Iter 50)	Getting better. Some trajectories succeed. Fewer failures.
Late (Iter 100)	Almost all trajectories lead to good outcomes. The policy has converged.

The Key Takeaway

The agent doesn’t need to know the rules of the world in advance. It just tries things, sees what worked, updates itself, and tries again. Over hundreds of iterations, this simple loop produces a very good policy.

1.2 Online Learning

Is this online learning only?

No. The slide specifically describes a **batch** update style — it says “collect a **batch** of data (trajectories)”, then update. That’s not online learning.

Here’s the distinction:

- **Online learning** — update the policy after **every single step** (or every transition). You don’t wait.
- **Batch learning** — collect a **full set of trajectories** first, then update the policy once using all of them. That’s what the slide shows.
- **Mini-batch** — a middle ground; collect a fixed number of episodes, update, repeat.

GPI as a **general framework** doesn’t lock you into one style. It just says “evaluate → improve → repeat.” How often you update is an implementation choice. That’s why both REINFORCE (batch, full episodes) and Actor-Critic methods (online or near-online, step-by-step) are both considered instances of GPI.

What happens in online learning?

In online learning, the agent updates the policy **after every single step** — no waiting, no batching.

The loop is:

1. Take one action in the environment
2. See the result (reward + next state)
3. Immediately update the policy
4. Take the next action using the already-updated policy

So the policy is changing **continuously** as the agent moves through the world. By the time the episode ends, the policy has already been updated dozens or hundreds of times.

The tradeoff: updates are fast and frequent, but each update is noisy because it’s based on just one data point instead of a whole batch of experience.

In online learning agent interacts with real environment directly. Right?

Not necessarily. “Online learning” just means **updating after every step** — it says nothing about whether the environment is real or simulated.

- In a **real environment** — yes, the agent takes a real action, gets a real reward, updates immediately.
- In a **simulated environment** — the agent still updates after every step, it’s still online learning. The environment just happens to be a simulator.

The word **online** refers to the **frequency of updates**, not the **type of environment**.

The distinction you might be thinking of is **Online (real environment)** vs. **Offline / Batch RL (replay buffer or pre-collected dataset)** — here “offline” means the

agent learns from a fixed dataset with no new interactions at all. But that’s a different axis than simulated vs. real.

1.3 Policy Without Q Values

Page 3 is titled “**Policy without Q values**” and introduces a new way of thinking about policies.

The Setup

So far in the course, GPI-based algorithms figured out a good policy by first computing $Q(s, a)$ — the expected reward of taking action a in state s . Once you know Q values well enough, you derive the policy from them (e.g., pick the action with the highest Q value). The slide says: what if we skip Q values entirely and just **directly build the policy itself**?

The New Idea: Parameterized Policy

Instead of storing Q values and deriving the policy from them, we represent the policy as a **mathematical function with adjustable knobs**. Those knobs are called **parameters**. We then tune the knobs directly until the policy gets good. This is called **parameterization** — you describe the policy using a set of numbers (the parameters), and learning means finding the right numbers.

The Equation

$$\pi(a \mid s, \vec{\theta}) = \Pr\{A_t = a \mid S_t = s, \vec{\theta}_t = \vec{\theta}\}$$

Symbol glossary:

Symbol	Meaning
--------	---------

π	The policy — the agent’s decision-making rule
a	A specific action the agent could take
s	The current state the agent is in
$\vec{\theta}$	The parameter vector — a list of numbers that defines the shape of the policy
d'	The number of parameters in $\vec{\theta}$ (its size/dimension)
$\Pr\{\cdot\}$	Probability of something happening
A_t	The action actually taken at time step t
S_t	The state the agent was in at time step t
t	A specific moment in time during the episode

The policy $\pi(a \mid s, \vec{\theta})$ gives you the **probability of choosing action** a , given that you’re in state s and your policy’s parameters are currently set to $\vec{\theta}$. The policy is no longer a lookup table — it’s a function. You plug in the state and the parameters, and it spits out a probability for each possible action. Learning now means **adjusting** $\vec{\theta}$ to make good actions more probable and bad actions less probable.

1.4 Parameterizing Policy

Page 4 is titled “**Parameterizing Policy**” and answers a practical question: if the policy is just a function with parameters, what rules must it follow, and what’s a concrete

example of such a function?

Two Rules Every Policy Must Obey

Since $\pi(a \mid s, \theta)$ outputs a **probability**, it has to follow the basic laws of probability. The slide gives two constraints.

Rule 1

$$\pi(a \mid s, \theta) \geq 0 \quad \text{for all } a \in \mathcal{A} \text{ and } s \in \mathcal{S}$$

Symbol glossary:

Symbol	Meaning
--------	---------

$\pi(a \mid s, \theta)$	Probability of taking action a in state s with parameters θ
$\mathcal{A}$	The set of all possible actions
$\mathcal{S}$	The set of all possible states

Every probability the policy outputs must be zero or positive. You can't have a negative probability — that makes no sense. This is just the most basic rule of probability.

Rule 2

$$\sum_{a \in \mathcal{A}} \pi(a \mid s, \theta) = 1 \quad \text{for all } s \in \mathcal{S}$$

Symbol glossary:

Symbol	Meaning
--------	---------

$\sum_{a \in \mathcal{A}}$	Add up over every possible action in $\mathcal{A}$
----------------------------	--

If you add up the probabilities of all possible actions in any given state, the total must equal 1. The agent has to do *something* — so all options together must account for 100% of the probability.

The Softmax Function — A Concrete Example

$$\pi(a \mid s, \theta) = \frac{e^{h(s,a,\theta)}}{\sum_{b \in \mathcal{A}} e^{h(s,b,\theta)}}$$

Symbol glossary:

Symbol	Meaning
--------	---------

$h(s, a, \theta)$	The preference score for action a in state s — a raw number produced by the parameter
$e^{(\cdot)}$	The exponential function — turns any number into a positive value, amplifies differences
b	A dummy variable representing each action in the sum (used to loop over all actions)
$\sum_{b \in \mathcal{A}} e^{h(s,b,\theta)}$	Total of exponential scores across all actions — used to normalize

First, compute a preference score $h(s, a, \theta)$ for every action. Then take e to the power of each score to make them all positive. Finally, divide each action's score by the total of all scores, so everything adds up to 1. The result is a valid probability for each action. Actions with higher preference scores get higher probabilities, but no action is ever completely ignored (its probability never hits exactly zero).

The Greedy Behavior Note

If the preference $h(s, a, \theta)$ of a particular action grows very large, the policy tends to act greedily. This means: if one action's score gets much bigger than all others, the softmax will assign it nearly 100% probability and ignore the rest. The policy stops exploring and just always picks that one action — it becomes **greedy**.

1.5 Preference Score and Feature Vectors

Who defines the preference score? How is it defined?

The parameters θ define it — specifically, $h(s, a, \theta)$ is a function that you design, and θ are the learnable weights inside that function.

How it's defined in practice

The most common choice is a **linear function**:

$$h(s, a, \theta) = \theta^\top x(s, a)$$

where $x(s, a)$ is a feature vector you hand-craft to describe the state-action pair, and θ is a weight vector the algorithm learns. The dot product gives a single score.

A more powerful choice is a **neural network** — you feed the state s in, and the network outputs one score per action. The weights of the network are θ , and they get updated through gradient descent.

So who decides the structure?

- **You (the designer)** choose the form of h — linear, neural network, etc.
- **The learning algorithm** (e.g., REINFORCE) then adjusts θ automatically over training to make good actions score higher and bad actions score lower.

The preference score has no inherent meaning on its own — it's just a raw number. What matters is the **relative difference** between scores across actions, because softmax only cares about which score is bigger, not the absolute values.

“ $x(s, a)$ is a feature vector you hand-craft”. What do you mean by hand-craft?

It means **you manually decide** what information to extract from the state-action pair and feed into the function.

Example — a robot navigating a grid

Say the state s is the robot's position, and action a is “move right.” You might hand-craft the feature vector as:

$$x(s, a) = \begin{bmatrix} \text{distance to goal} \\ \text{distance to wall} \\ 1 \text{ if action} = \text{right, else } 0 \\ 1 \text{ if action leads to open space, else } 0 \end{bmatrix}$$

You — the human — decided that these four pieces of information matter. You wrote that logic manually. That’s what “hand-crafted” means.

Why is this a problem?

- You might miss important features
- You might include useless ones
- For complex problems (images, text, games), it’s nearly impossible to hand-craft good features

That’s exactly why neural networks replaced hand-crafted features

A neural network takes the raw state (e.g., raw pixels) and **learns its own internal features automatically** from experience. You don’t decide what matters — the network figures it out. The weights θ inside the network serve the same role as the hand-crafted $x(s, a)$, but nothing is manually designed. So hand-crafting was the old way. Neural networks are the modern way.

1.6 The RL Objective

Page 6 is titled “**The RL Objective**” — it formally states what the agent is actually trying to maximize, and how to estimate it in practice.

The Big Goal

The agent wants to find the best possible parameters θ for its policy. “Best” means: the parameters that produce the highest total reward on average across all possible trajectories.

Equation 1 — Finding the Best Parameters

$$\vec{\theta}^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [r(\tau)]$$

Symbol glossary:

Symbol	Meaning
$\vec{\theta}^*$	The optimal (best) parameter vector — what we’re searching for
$\arg \max_{\theta}$	“Give me the value of θ that makes the thing inside as large as possible”
τ	A trajectory — one full sequence of states and actions from start to finish
$p_{\theta}(\tau)$	The probability of trajectory τ happening under policy θ
$\mathbb{E}_{\tau \sim p_{\theta}(\tau)}[\cdot]$	The average value of something, taken over all trajectories the policy generates
$r(\tau)$	The total reward collected along trajectory τ

We want the parameters θ^* that make the **average total reward across all trajectories** as high as possible. We're not optimizing for one lucky run — we want good performance on average.

Equation 2 — Expanding the Reward

$$= \arg \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\underbrace{\sum_t r(s_t, a_t)}_{J(\theta)} \right]$$

Symbol glossary:

Symbol Meaning

$\sum_t$ Add up over every time step t in the trajectory

$r(s_t, a_t)$ The reward received at time step t , for being in state s_t and taking action a_t

$J(\theta)$ The objective function — the single number we want to maximize; our score for how good

The total reward of a trajectory $r(\tau)$ is just the sum of all the small rewards collected at each individual step. $J(\theta)$ is the name we give to the expected value of that sum — it's our score for how good a given θ is.

Equation 3 — How to Actually Compute It

$$J(\theta) \approx \frac{1}{N} \sum_i \sum_t r(s_{i,t}, a_{i,t})$$

Symbol glossary:

Symbol Meaning

$\approx$ Approximately equal to

N Number of trajectories (sample runs) collected

$\sum_i$ Add up over each trajectory i (each run)

$\sum_t$ Add up over each time step within that trajectory

$r(s_{i,t}, a_{i,t})$ Reward at time step t in trajectory i

$\frac{1}{N}$ Divide by number of trajectories to get the average

You can't compute the true expectation $J(\theta)$ exactly — that would require running infinitely many trajectories. Instead, you run N trajectories, add up all the rewards collected across all time steps in all trajectories, and divide by N to get the average. This average is a good enough approximation of the true $J(\theta)$.

The Diagram on the Right

This is exactly what the approximation looks like in practice. The dot on the left is the starting state. Each curved arrow is one trajectory — one attempt by the agent. Some end in red X (bad outcome, low reward), some in yellow or green checkmark (good or great outcome). You run N of these, average the total rewards, and that gives you your estimate of $J(\theta)$.

1.7 Pages 7–8: Deriving the Gradient

Page 7 — Gradient of the RL Objective

Quick recap. Remember from page 6: the agent’s goal is to maximize $J(\theta)$, the average total reward it gets by following its policy. To improve the policy, we need to compute the **gradient** of $J(\theta)$ — basically, the direction in which to nudge the policy knobs (θ) to get more reward.

Equation 1 — Rewriting the objective as an integral.

$$J(\theta) = E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(s_t, a_t) \right] = \int p_\theta(\tau) r(\tau) d\tau$$

Symbol	Meaning
--------	---------

$J(\theta)$	The score of the policy — total expected reward
-------------	---

θ	The policy’s knobs (parameters)
----------	---------------------------------

τ	A trajectory — a full playthrough: $s_1, a_1, s_2, a_2, \dots$
--------	--

$p_\theta(\tau)$	Probability of that trajectory happening under policy θ
------------------	--

$r(\tau)$	Total reward collected along that trajectory
-----------	--

$\int \dots d\tau$	“Sum over all possible trajectories”
--------------------	--------------------------------------

This is just rewriting the expected reward in math notation. Instead of saying “average reward over many runs,” we write it as an integral — summing $p_\theta(\tau) \cdot r(\tau)$ over every possible trajectory. The integral is mathematically equivalent to the expectation, which sets us up for taking the gradient.

Equation 2 — Taking the gradient (the log-derivative trick).

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau) r(\tau) d\tau = \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) r(\tau) d\tau = E_{\tau \sim p_\theta} [\nabla_\theta \log p_\theta(\tau) r(\tau)]$$

Symbol	Meaning
--------	---------

∇_θ	“How much does this change when we tweak θ ?” (the gradient)
-----------------	---

$\nabla_\theta p_\theta(\tau)$	Gradient of the trajectory probability — hard to work with directly
--------------------------------	---

$\nabla_\theta \log p_\theta(\tau)$	Gradient of the <i>log</i> of the trajectory probability — much easier
-------------------------------------	--

We want the gradient of $J(\theta)$, but differentiating $p_\theta(\tau)$ directly is messy because it involves the unknown environment dynamics. The **log-derivative trick** says: $p_\theta(\tau) \cdot \nabla_\theta \log p_\theta(\tau) = \nabla_\theta p_\theta(\tau)$. This lets us swap the hard term out and rewrite the whole gradient as an **expectation** — which means we can estimate it just by running the policy and averaging, no integrals needed.

The key identity.

$$p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) = p_\theta(\tau) \cdot \frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} = \nabla_\theta p_\theta(\tau)$$

This is just the chain rule of calculus: the derivative of $\log(x)$ is $\frac{1}{x}$. Multiplying $p_\theta(\tau)$ on both sides cancels it out, proving the substitution above is valid.

Page 8 — Expanding the Gradient into Something Computable

Now the slide answers: “Okay, but how do we actually **compute** $\nabla_\theta \log p_\theta(\tau)$?”

Equation 3 — What is $p_\theta(\tau)$?

$$p_\theta(\tau) = p(s_1) \prod_{t=1}^T \pi_\theta(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

Symbol	Meaning
$p(s_1)$	Probability of starting in state s_1 (set by the environment, not the agent)
$\prod_{t=1}^T$	Multiply together across all time steps $t = 1$ to T
$\pi_\theta(a_t s_t)$	Policy’s probability of picking action a_t when in state s_t
$p(s_{t+1} s_t, a_t)$	Environment’s probability of moving to s_{t+1} given state and action

A trajectory’s probability is the product of every decision and every world transition along the way. Think of it like the probability of a specific path through a maze — it’s the starting probability times the chance you took each turn times the chance the world responded the way it did, multiplied across every step.

Equation 4 — Expanding $\log p_\theta(\tau)$.

$$\nabla_\theta \left[\log p(s_1) + \sum_t \log \pi_\theta(a_t | s_t) + \log p(s_{t+1} | s_t, a_t) \right]$$

Taking the log of a product turns it into a sum (since $\log(ABC) = \log A + \log B + \log C$). The critical insight is that $\log p(s_1)$ and $\log p(s_{t+1} | s_t, a_t)$ do not contain θ at all — they are controlled by the environment, not the policy. So their gradients are **zero** and they vanish. Only $\log \pi_\theta(a_t | s_t)$ survives.

Equation 5 — The practical gradient formula.

$$\nabla_\theta J(\theta) = E_{\tau \sim p_\theta} \left[\left(\sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \right) \left(\sum_{t=1}^T r(s_t, a_t) \right) \right]$$

Symbol	Meaning
$\sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t s_t)$	How to nudge the policy to make each action more or less likely
$\sum_{t=1}^T r(s_t, a_t)$	Total reward collected during the trajectory

The gradient is the product of two sums — “how much did each action’s probability change when we tweak θ ?” multiplied by “how much total reward did we get?” If a trajectory got high reward, we push θ in the direction that made those actions more

likely. If it got low reward, we push the other way. Crucially, the environment dynamics $p(s_{t+1} \mid s_t, a_t)$ are **completely gone** — we only need the policy and the rewards.

Equation 6 — The sample estimate (how we actually compute it).

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} \mid s_{i,t}) \right) \left(\sum_{t=1}^T r(s_{i,t}, a_{i,t}) \right)$$

Symbol	Meaning
N	Number of sampled trajectories (rollouts)
i	Index for each sampled trajectory
$\frac{1}{N} \sum_i$	Average over all N rollouts
$a_{i,t}, s_{i,t}$	Action and state at time t in the i -th trajectory

Since we cannot compute an expectation over *all* possible trajectories, we approximate it by running the policy N times, computing the gradient formula for each run, and averaging. This is the same idea as estimating the average of a die roll by rolling it many times — the more rollouts, the better the estimate. This is the actual algorithm you would implement in code.

1.8 Page 9: The Full REINFORCE Algorithm

This page puts everything from pages 7 and 8 together into one complete, runnable algorithm. It has a name: **REINFORCE**, also called the **vanilla policy gradient**. “Vanilla” just means the plain, basic version — no fancy extras yet.

Think of it like a recipe with 4 steps that you repeat over and over until the agent gets good.

Step 1 — Collect experience.

Sample $\{\tau^i\}$ from $\pi_{\theta}(a_t \mid s_t)$

Symbol	Meaning
$\{\tau^i\}$	A collection of trajectories (multiple full playthroughs), indexed by i
$\pi_{\theta}(a_t \mid s_t)$	The current policy — given state s_t , it picks action a_t

Run your current policy in the environment multiple times and record what happened each time — which states you visited, which actions you took, and what rewards you got. This is just the agent “playing the game” several times and taking notes.

Step 2 — Compute the gradient.

$$\nabla_{\theta} J(\theta) \approx \sum_i \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i \mid s_t^i) \right) \left(\sum_{t=1}^T r(s_t^i, a_t^i) \right)$$

Symbol	Meaning
$\nabla_{\theta} J(\theta)$	The gradient — which direction to push the policy knobs
$\sum_i$	Sum (add up) over all collected trajectories
$\nabla_{\theta} \log \pi_{\theta}(a_t^i s_t^i)$	How much action a_t in trajectory i would change if we tweaked θ
$\sum_{t=1}^T r(s_t^i, a_t^i)$	Total reward collected in trajectory i

For each recorded trajectory, multiply “how sensitive was the policy to its own choices” by “how much reward did it get?” Add this up across all trajectories. Trajectories with high reward push the gradient hard in a direction that makes those actions more likely; trajectories with low reward push in the opposite direction. This is the signal that tells us how to improve.

Step 3 — Update the policy.

$$\vec{\theta} = \vec{\theta} + \alpha \nabla_{\theta} J(\theta)$$

Symbol	Meaning
$\vec{\theta}$	The policy’s parameters (knobs) — a vector of numbers
α	Learning rate — controls how big a step to take
$\nabla_{\theta} J(\theta)$	The gradient computed in Step 2

Nudge the policy parameters θ in the direction of the gradient by a small amount controlled by α . Note it is a **plus** sign, not minus — because we are doing **gradient ascent** (climbing toward higher reward), unlike regular gradient descent used in supervised learning which minimizes a loss. A large α means big jumps (risky); a small α means careful, slow improvement.

Step 4 — Repeat until convergence.

Loop back to Step 1 with the newly updated policy. Collect new data, compute a new gradient, update again. Keep going until the policy stops improving noticeably — that is convergence.

The big picture. The diagram on the slide shows this loop: collect experience by running the policy, then improve the policy using that data, then repeat. REINFORCE looks at all collected trajectories, figures out which actions led to good outcomes, and makes those actions more likely next time. It is literally trial and error, formalized into math.

1.9 Page 10: Understanding What the Gradient Actually Does

This page is not introducing new math. It is stepping back and asking: **what does this gradient formula actually mean in plain English?** It is the intuition behind everything derived on pages 7–9.

The gradient (same formula, new lens).

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \right) \left(\sum_{t=1}^T r(s_t^i, a_t^i) \right)$$

Symbol	Meaning
$\frac{1}{N} \sum_i$	Average over all N collected trajectories
$\nabla_{\theta} \log \pi_{\theta}(a_t^i s_t^i)$	Direction to push θ to make action a_t in trajectory i more likely
$\sum_{t=1}^T r(s_t^i, a_t^i)$	Total reward of trajectory i — the weight

The total reward of a trajectory acts as a **multiplier** on the gradient. If a trajectory earned high reward, its gradient contribution gets multiplied by a big positive number — meaning those actions get strongly encouraged. If the reward was negative, the multiplier flips the sign and those actions get discouraged. The gradient is literally a weighted vote from every trajectory.

The three key points from the slide.

1. **Increase the likelihood of actions you took in high-reward trajectories.** If a playthrough went well, assume the actions you took during it were probably good. Make those actions more likely in the future.
2. **Decrease the likelihood of actions you took in negative-reward trajectories.** If a playthrough went badly, assume those actions were probably bad. Make them less likely going forward.
3. **Do more of the good stuff, less of the bad stuff.** That is the whole algorithm in one sentence. The math just makes this idea precise and automatic.

The diagram. The picture shows several trajectories starting from the same point. Some end in failure and one ends in success. After running REINFORCE, the policy will shift probability mass toward whatever sequence of actions led to the successful path, and away from the actions that led to failure. Over many iterations, the successful path becomes more and more likely.

Why this page matters. Everything on pages 7 and 8 was dense derivation. This page is the payoff — it tells you that all that math ultimately boils down to one dead-simple idea: **reward good behavior, punish bad behavior, repeat.** REINFORCE is just that idea, expressed precisely enough for a computer to execute.

1.10 Pages 11–12: A Problem with the Gradient and the Reward-to-Go Fix

Page 11 — Quiz: A Problem With the Gradient

This page gives a concrete worked example to expose a flaw hiding inside the gradient formula. The task: **teach a robot to walk forward.**

The reward is simple:

$$r(s, a) = \text{forward velocity of the robot (negative if it goes backwards)}$$

The agent runs 5 trajectories (τ^1 through τ^5):

Trajectory	What happened	Reward
τ^1	Falls backwards	Negative
τ^2	Falls forward	Small positive
τ^3	Stands still	Zero
τ^4	One small step forward, then falls backwards	Mixed
τ^5	One large step backwards, then small step forwards	Negative overall

The question the slide asks: given these 5 trajectories and the gradient formula from page 9, **what will the gradient actually encourage the policy to do?**

The answer: **it will encourage the robot to fall forward — not to actually take a step forward.**

Why? The flaw explained.

τ^2 (falls forward) gets a positive total reward because at least it was moving in the right direction for a moment. τ^4 (takes a step, then falls backward) might net out to a small or zero total reward because the backward fall cancels the forward step.

So when the gradient formula multiplies each trajectory's actions by its **total reward**, τ^2 gets a stronger positive push than τ^4 . The gradient is essentially saying: “falling forward is better than stepping forward and then falling backward” — which is wrong for the actual goal of learning to walk.

This is a **credit assignment problem**: the total reward of a whole trajectory is being used to judge every single action inside it, even actions that happened early and had nothing to do with the final outcome.

Page 12 — Improving the Gradient (Reward-to-Go)

The key insight.

Policy behavior at time t does not affect rewards at time $t' < t$.

In plain English: **your action right now cannot change what already happened in the past.** If you fall at step 5, that cannot hurt or help your reward from steps 1–4. Those are already done. So it is unfair — and noisy — to judge an action at time t by the rewards that came **before** it. We should only hold an action responsible for the rewards that come **after** it (including the current moment). These future rewards are called the **reward-to-go**.

Old gradient (the flawed one).

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \right) \left(\sum_{t=1}^T r(s_t^i, a_t^i) \right)$$

The second sum is the **total reward of the entire trajectory** — it judges every action by the sum of all rewards, past and future.

New gradient (the fix).

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \left(\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i) \right)$$

Symbol	Meaning
$\sum_{t=1}^T$	Outer sum now goes over each individual timestep t , not just trajectories
$\nabla_{\theta} \log \pi_{\theta}(a_t^i s_t^i)$	Same as before — how to nudge θ for action a_t
$\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i)$	Reward-to-go: rewards from time t onward only

Instead of judging action a_t by the total reward of the whole episode, we only judge it by the rewards it could have actually influenced — everything from time t to the end. This is fairer and makes the gradient less noisy, because past rewards that had nothing to do with action a_t are no longer polluting its signal.

Back to the walking robot. With reward-to-go, τ^4 (step forward, then fall back) gets properly evaluated. The forward step at time $t = 1$ gets credited with the rewards from $t = 1$ onward — which include the positive forward velocity from that step. The backward fall at time $t = 2$ only gets blamed for rewards from $t = 2$ onward. The gradient now correctly separates the good action from the bad one within the same trajectory, instead of blurring them together under one total reward number.

1.11 Quiz: What the Gradient Does — The Walking Robot (Page 13)

The Setup

A humanoid robot is learning to walk. The reward $r(s, a)$ is the robot's forward speed. Move forward $\rightarrow$ positive reward. Move backward $\rightarrow$ negative reward.

Step 1 — Collect sample episodes

The robot tries 4 different episodes under the current policy $\pi_{\theta}(a_t | s_t)$:

Episode	What happened	Reward
τ^1	Falls forwards	Positive (moved forward)
τ^2	Slowly stumbles forwards	Positive
τ^3	Steadily walks forwards	Positive
τ^4	Runs forwards	Positive

All four got **positive reward** because all of them moved forward.

Step 2 — Compute the gradient

$$\nabla_{\theta} J(\theta) \approx \sum_i^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \left(\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i) \right)$$

- $\nabla_{\theta} J(\theta)$ — the gradient of the total reward with respect to the policy parameters; the “which direction to improve” signal
- $\sum_i^N$ — sum over all N sampled episodes

- $\sum_{t=1}^T$ — sum over every time step in an episode
- $\nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i)$ — how much each action’s probability should shift
- $\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i)$ — total future reward collected from time step t onwards (the “return”)

For every action taken in every episode, multiply “how much this action’s probability should change” by “how much reward followed from this point.” Average this over all episodes to get the gradient — the direction to nudge the policy.

The Surprising Answer

Since **all four episodes got positive reward**, the gradient gives a positive push to **all of them** — including the falling episode (τ^1). The gradient cannot tell the difference between “fall forward” and “run forward.” It just sees “positive reward $\rightarrow$ do more of that.” So the policy gets nudged toward falling forward, which is clearly not the ideal walking behavior.

This is the core problem: **policy gradient is high-variance** — its gradient estimates are noisy and sensitive to the overall scale of rewards. If everything gets positive reward, everything gets reinforced, even bad behavior.

1.12 Introducing the Baseline (Page 14)

The Core Idea

Instead of reinforcing an action whenever the reward is positive, reinforce it only when the reward is **better than average**. This is done by subtracting a **baseline** b from the reward.

The Modified Gradient Formula

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}} [\nabla_{\theta} \log p_{\theta}(\tau) \cdot (r(\tau) - b)]$$

- $\nabla_{\theta} J(\theta)$ — gradient of the objective (direction to improve policy)
- $E_{\tau \sim p_{\theta}}[\cdot]$ — average over all episodes sampled from the current policy
- $\nabla_{\theta} \log p_{\theta}(\tau)$ — how much the probability of this episode’s actions should shift
- $r(\tau)$ — total reward of episode τ
- b — the baseline (a reference number we subtract)
- $(r(\tau) - b)$ — how much better or worse this episode was compared to the baseline

Instead of asking “did this episode get good reward?”, we now ask “did this episode do **better than the baseline**?” If yes, increase the probability of those actions. If no, decrease it. The baseline acts like a reference point — a “passing grade.”

What is the baseline?

The simplest choice is the **average reward** across all sampled episodes:

$$b = \frac{1}{N} \sum_i r(\tau)$$

- N — total number of sampled episodes
- $r(\tau)$ — reward of each episode
- b — their simple average

This is just computing the average score across all your robot’s attempts.

Applying it to the walking example

Say the average reward across the 4 episodes is some value b . Now:

- “Running forward” (τ^4) scores well above $b \rightarrow (r - b) > 0 \rightarrow$ get a **positive push** (do more of this)
- “Falling forward” (τ^1) scores below $b \rightarrow (r - b) < 0 \rightarrow$ get a **negative push** (do less of this)

Now the gradient correctly discourages falling and encourages running, even though both had positive raw rewards.

Does subtracting the baseline break anything?

No — and this is proved at the bottom of the slide. Subtracting a constant baseline leaves the **expected gradient unchanged** (it is unbiased). The proof shows:

$$E[\nabla_{\theta} \log p_{\theta}(\tau) \cdot b] = b \nabla_{\theta} \int p_{\theta}(\tau) d\tau = b \nabla_{\theta} \cdot 1 = 0$$

The key step: $\int p_{\theta}(\tau) d\tau = 1$ because all probabilities must sum to 1. The gradient of a constant (1) is zero. So the baseline contributes **zero** to the expected gradient — it does not bias the direction of learning at all. But it **reduces variance** significantly, because individual episode rewards are now measured relative to the average instead of in absolute terms. Smaller swings in the gradient = more stable learning.

1.13 Quiz: Sparse Rewards — Jacket Folding (Page 15)

The Setup

A robot is learning to fold a jacket. The reward has only three possible values:

Outcome	Reward
Doesn’t touch the jacket	0
Folded with wrinkles	0.5
Neatly folded	1

Step 1 — Sample episodes

Episode	What happened	Reward
τ^1	Doesn't touch the jacket	0
τ^2	Folds only the sleeves	≈ 0
τ^3	Flattens but doesn't fold	0
τ^4	Folds the jacket	1

Three out of four episodes got zero reward. Only τ^4 succeeded.

Step 2 — Compute the gradient with baseline

$$\nabla_{\theta} J(\theta) \approx \sum_i^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \left(\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i) - b \right)$$

- $\nabla_{\theta} J(\theta)$ — the gradient; tells us which direction to nudge the policy
- $\nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i)$ — how much each action's probability should shift
- $\sum_{t'=t}^T r(s_{t'}^i, a_{t'}^i)$ — total future reward from step t onward in episode i
- b — the baseline (average reward across all episodes, which is close to 0 here)

For each action taken in each episode, push that action's probability up or down based on how much better-than-average the outcome was. Subtract the baseline b so only above-average results get reinforced.

What happens in practice here?

Since most episodes scored 0 and the baseline $b \approx 0$, we get $(r - b) \approx 0$ for τ^1, τ^2, τ^3 — their gradient contributions are essentially **zero**. Only τ^4 , which scored 1, gives a meaningful signal.

So the gradient correctly points toward folding behavior — but the problem is it barely received any signal at all. Three out of four episodes contributed almost nothing to learning. This is called the **sparse reward problem**: when success is rare, most of your data is useless, the gradient is nearly flat most of the time, and learning is painfully slow and noisy. The baseline helped in the walking example (page 13) but did not fully fix this new problem.

1.14 Vanilla Policy Gradient and the On-Policy Problem (Page 16)

The Full Algorithm

1. **Sample** episodes $\{\tau^i\}$ from the current policy $\pi_{\theta}(a_t | s_t)$
2. **Compute** the gradient $\nabla_{\theta} J(\theta)$ using those episodes
3. **Update** the parameters
4. **Repeat** until the policy stops improving (convergence)

The update equation (Step 3):

$$\vec{\theta} = \vec{\theta} + \alpha \nabla_{\theta} J(\theta)$$

- $\vec{\theta}$ — the policy’s parameters (all the knobs controlling what actions the policy picks)
- α — the learning rate; how big a step to take (too big $\rightarrow$ overshoot, too small $\rightarrow$ crawl)
- $\nabla_{\theta} J(\theta)$ — the gradient; the direction that increases total reward

Take the current parameters $\vec{\theta}$, and nudge them in the direction of the gradient by a small amount α . Repeat. This is plain gradient ascent — climbing uphill on the reward surface one step at a time.

The Big Problem: On-Policy Learning

Vanilla policy gradient is **on-policy**, meaning the episodes used to compute the gradient in Step 2 must have been collected by the **exact current version of the policy**. Once you update θ in Step 3, those episodes are immediately outdated and thrown away. You can never reuse them.

This creates a painful cycle: collect data $\rightarrow$ compute gradient $\rightarrow$ update policy $\rightarrow$ **throw all data away** $\rightarrow$ collect new data $\rightarrow \dots$

Every single gradient step requires a fresh batch of episodes. If collecting data is slow (a real robot trying things in the physical world, or an expensive simulation), this wastes enormous amounts of time.

The natural question is: **what if we want to reuse data collected by an older version of the policy?** The answer is **off-policy RL using importance sampling** — a mathematical correction that lets you reuse old data by accounting for the fact that the old policy and the new policy chose actions differently. This is what the next slides address.

1.15 Off-Policy Policy Gradient (Page 17)

The Big Idea: Importance Sampling

Imagine you want to know what score a student would get on a different exam, but you only have their answers from the original exam. You can still estimate it — but you have to *adjust* each answer’s weight based on how different the two exams are. That adjustment factor is called the **importance sampling ratio**.

Same idea here: we collected episodes using the *old* policy θ , but we want to compute the gradient for a *new* (updated) policy θ' . We correct for this mismatch by multiplying by the ratio of the two policies’ probabilities.

Equation 1 — The Exact Off-Policy Gradient

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\prod_{t=1}^T \frac{\pi_{\theta'}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)} \left(\sum_{t=1}^T \nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t) \right) \left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) - b \right) \right]$$

- θ' — the **new** policy’s parameters (the one we are trying to improve)
- θ — the **old** policy’s parameters (the one that actually collected the data)

- $\mathbb{E}_{\tau \sim p_{\theta}(\tau)}[\cdot]$ — average over episodes collected using the **old** policy θ
- $\prod_{t=1}^T \frac{\pi_{\theta'}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)}$ — the **importance sampling correction**: a product of ratios, one per time step; each ratio asks “how much more or less likely would the new policy have been to take this same action?”
- $\nabla_{\theta'} \log \pi_{\theta'}(a_t | s_t)$ — how much to shift each action’s probability under the new policy
- $\sum_{t'=t}^T r(s_{t'}, a_{t'}) - b$ — future reward from step t onward, minus the baseline

This is the REINFORCE gradient formula from before, but now with a correction factor bolted on — the big product in front. It uses data from the old policy, but multiplies each episode’s contribution by how differently the new policy would have behaved throughout that episode, step by step.

The Problem with Equation 1

The correction factor is a **product of T ratios** — one ratio multiplied together for every time step in the episode. If the episode is long (T is large), this product becomes unstable:

- If each ratio is slightly less than 1 $\rightarrow$ the product shrinks toward **zero** (gradient vanishes — no learning signal)
- If each ratio is slightly greater than 1 $\rightarrow$ the product explodes toward **infinity** (gradient blows up — wild, unstable updates)

This is like compounding interest but in a dangerous way. Multiplying together 100 numbers that are each 0.9 gives $0.9^{100} \approx 0.000027$ — basically nothing.

Equation 2 — Switching from Trajectories to Timesteps

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(s_{i,t}, a_{i,t})}{\pi_{\theta}(s_{i,t}, a_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) - b \right)$$

- $\frac{\pi_{\theta'}(s_{i,t}, a_{i,t})}{\pi_{\theta}(s_{i,t}, a_{i,t})}$ — the ratio of how likely it was to **be in state s and take action a** under the new policy versus the old policy (a single ratio per timestep, not a product over all steps)
- $s_{i,t}, a_{i,t}$ — the state and action at time step t in episode i
- Everything else is the same as Equation 1

Instead of one giant product across the whole trajectory, we now use a **single ratio per time step** — comparing the new and old policy just at that one moment. This avoids the vanishing/exploding problem because there is no long chain of multiplications. The slide says this is “much less likely to explode/vanish.” However, the ratio $\frac{\pi_{\theta'}(s_{i,t}, a_{i,t})}{\pi_{\theta}(s_{i,t}, a_{i,t})}$ is the probability of both being in state s *and* taking action a . The state part is hard to compute — we do not know exactly how often the new policy would visit each state, because state visits depend on the full history of decisions. It is “hard to measure.”

Equation 3 — The Final Practical Form

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_{\theta}(a_{i,t} | s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \left(\sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) - b \right)$$

- $\frac{\pi_{\theta'}(a_{i,t}|s_{i,t})}{\pi_{\theta}(a_{i,t}|s_{i,t})}$ — the ratio of **just the action probabilities**: given that we are already in state s , how much more or less likely is the new policy to pick action a compared to the old policy? The state part is dropped entirely.
- Everything else is the same as Equation 2

The state visitation ratio (the hard-to-measure part) is dropped, and only the action probability ratio is kept. This is an approximation — we are assuming the new and old policy visit states at roughly the same frequency — but it makes the formula practical. Both $\pi_{\theta'}(a | s)$ and $\pi_{\theta}(a | s)$ are just policy outputs that can be computed directly by plugging the state into the policy network.

Putting It All Together

The three equations represent a progression of trade-offs:

Equation	Correction factor	Explode/vanish?	Easy to compute?
Eq. 1	Product over all T steps	Yes, badly	Yes
Eq. 2	Single state-action ratio per step	No	No (state part is hard)
Eq. 3	Single action-only ratio per step	No	Yes

Equation 3 is the final usable formula. It lets you reuse data from an old policy, corrects for the mismatch using a simple per-step ratio, and is fully computable. This is the foundation for **off-policy** algorithms that are far more data-efficient than vanilla policy gradient.

1.16 Off-Policy Policy Gradient: Full Algorithm (Page 18)

The Algorithm (same three steps, different loop)

1. Sample episodes $\{\tau^i\}$ from policy $\pi_{\theta}(a_t | s_t)$
2. Compute $\nabla_{\theta} J(\theta)$ using those episodes
3. Update: $\vec{\theta} = \vec{\theta} + \alpha \nabla_{\theta} J(\theta)$

The update equation:

$$\vec{\theta} = \vec{\theta} + \alpha \nabla_{\theta} J(\theta)$$

- $\vec{\theta}$ — the policy's parameters (all the internal settings that control behavior)
- α — the learning rate; how big each step is
- $\nabla_{\theta} J(\theta)$ — the gradient; the direction that increases reward

Nudge the parameters in the direction that increases reward, scaled by the step size α . This is the same update as vanilla policy gradient.

What is actually different: the loop

The slide shows two arrows on the algorithm:

- **Red arrow** (on-policy loop): After step 3, loop back all the way to step 1 — collect **new** data before doing anything else.
- **Orange arrow** (off-policy loop): After step 3, loop back only to step 2 — recompute the gradient on the **same old data** using the importance sampling correction from page 17, and update again.

In vanilla policy gradient, after one update you must throw away all your data and go collect more. In off-policy policy gradient, you can reuse the same batch and do **multiple gradient steps** on it — because the importance sampling ratio in the gradient formula already corrects for the fact that the policy has changed since the data was collected.

Think of it like studying for an exam: on-policy is like solving one practice problem and then immediately going to get a new set — very inefficient. Off-policy is like solving the same set of practice problems multiple times, learning more from them each pass through.

1.17 The KL Divergence Constraint (Page 19)

The Problem

When you take many gradient steps on the same batch of old data, the new policy θ' can drift very far from the old policy θ that collected the data. But the importance sampling correction only works well when the two policies are *similar*. Once they diverge significantly:

- The old data no longer reflects the states the new policy would actually visit
- The gradient estimate becomes increasingly wrong
- You might make a bad update that seriously hurts the policy's performance — and you might not even realize it until it is too late

It is like navigating using an old map. A slightly outdated map is fine. But if the roads have completely changed, the map will lead you off a cliff.

The Fix — Constrain How Much the Policy Can Change

$$\mathbb{E}_{s \sim \pi_\theta} [D_{\text{KL}}(\pi_{\theta'}(\cdot | s) \parallel \pi_\theta(\cdot | s))] \leq \delta$$

- $\mathbb{E}_{s \sim \pi_\theta} [\cdot]$ — average over states that the **old** policy θ visited; we are measuring the difference in the situations the old policy actually experienced
- $D_{\text{KL}}(\cdot \parallel \cdot)$ — **KL divergence**: a mathematical ruler for measuring how different two probability distributions are; it equals 0 when they are identical, and grows larger as they diverge
- $\pi_{\theta'}(\cdot | s)$ — the **new** policy's action choices at state s (a full probability distribution over actions)
- $\pi_\theta(\cdot | s)$ — the **old** policy's action choices at state s
- δ — a small threshold (your “budget” for how much the policy is allowed to change); typically a small number like 0.01

On average across all the states the old policy visited, the new policy must not be “too different” from the old one. “Too different” is measured by KL divergence, and the constraint says it must stay at or below the budget δ . This limits how large each policy update can be, keeping the new policy close enough to the old one that the recycled data remains trustworthy.

Why This Matters

This constraint is the core idea behind **Trust Region Policy Optimization (TRPO)** — one of the most important algorithms in modern RL. The name comes from this: you trust the old data only within a certain region of policy space. Inside that region (where new and old policies are close), the gradient estimate is reliable. Outside it, you cannot trust the old data anymore and must stop.

A simpler version of this same idea later became **PPO (Proximal Policy Optimization)**, which enforces a similar limit without the full math of a trust region, and is today one of the most widely used RL algorithms in practice — including in training large language models like the ones behind ChatGPT.